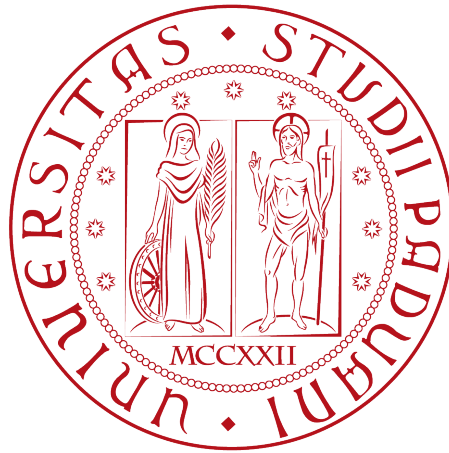


Università degli Studi di Padova

DEPARTMENT OF MATHEMATICS

MASTER DEGREE IN COMPUTER SCIENCE



FORMAL METHODS FOR CYBERPHYSICAL SYSTEMS

AY 2024-25

ALTERNATIVE PROJECT: TECHNICAL REPORT

Jacopo Angeli
2130757

JULY 30, 2025

Verification of reactivity properties

Problem overview

The goal is to develop an algorithm that can verify a specific class of LTL properties called reactivity properties. These are formulas of the form

$$\Box\Diamond f \rightarrow \Box\Diamond g$$

where f and g are Boolean formulas without temporal operators.

The formula asserts that if the condition f keeps becoming true infinitely often, then the condition g must also occur infinitely often. The solution presented checks whether its negation holds rather than directly proving the implication.

The negation of the formula $\Box\Diamond f \rightarrow \Box\Diamond g$ is $\Box\Diamond f \wedge \Box\Diamond\neg g$ that follows from the logical equivalence $f \rightarrow g \equiv \neg f \vee g$, hence $\neg(f \rightarrow g) \equiv f \wedge \neg g$. Intuitively the original formula is violated if there is an execution in which f continues to occur infinitely often while g permanently stops occurring. Instead of proving the implication directly, the algorithm searches for a counterexample to the implication by checking the negation,

The solution searches for a cycle in the state space where f is satisfied and g is not. Finding such cycle is sufficient to prove the property is not valid, and was partially covered during lectures. If the formula $\Box\Diamond f \wedge \Box\Diamond\neg g$ is satisfiable, the witness is build identifying the loop head, then constructing the loop body, builds the prefix that leads to it from the initial states, and finally combining everything into a concrete trace.

Symbolic reactivity check function

Each portion of the function is briefly introduced with a comment about the logic behind it, and followed by the corresponding code snippet and some consideration on its correctness. Comments and integration to the pseudo-code presented during the course are highlighted in `gray` and `red` respectively.

Sanity check and model preparation

The function begins by parsing the input specification to filter non reactivity properties returning `None`. In case of a valid reactivity formula f and g are extracted, and the negation of g is constructed. The finite state machine is then prepared in BDD form, and both f and $\neg g$ are converted into their corresponding BDD representations.

```
parse_result = parse_react(spec)
if parse_result is None:
    return None

f, g = parse_result
ng = pynusmv.prop.not_(g)
```

```
fsm_bdd = pynusmv.glob.prop_database().master.bddFsm
f_bdd = spec_to_bdd(fsm_bdd, f)
ng_bdd = spec_to_bdd(fsm_bdd, ng)
```

Reachable states

The usage of `reachable_states` from the `pynusmv` library wasn't allowed for this project, hence the solution implements verbatim the symbolic BFS algorithm presented in Part 7 of the course. The algorithm explores the state space starting from the initial region, by definition reachable and incrementally expanding the set of reachable states by computing successors and discarding already visited states until no new states are found.

```
reach_bdd = fsm_bdd.init
new_bdd = fsm_bdd.post(reach_bdd)
while fsm_bdd.count_states(new_bdd) > 0:
    new_bdd = fsm_bdd.post(new_bdd).diff(reach_bdd)
    reach_bdd = reach_bdd.union(new_bdd)
```

Symbolic repeatability check

The repeatability check essentially computes a fixed point over the reachable state space considering only the transitions that satisfy the input formula, returning a set that remains unchanged under repeated application of the pre-image operator. In each iteration it identifies the set of states that can reach the previous set in a single step, (i.e. their pre-image) starting from the portion of initial states that satisfies $f \wedge \neg g$.

If the set of states stabilizes, meaning it does not change after applying the pre-image operator, then the property is repeatable and the algorithm returns `(True, None)`. If instead the set keeps changing, it means that there exists a loop in which f is satisfied and g is not, hence the property is not repeatable and a witness can be constructed. The witness construction is treated in the next subsection, hence the following code is the main body of the repeatability check where the witness construction starting point is indicated by a comment:

```
recur_bdd = reach_bdd.intersection(f_bdd).intersection(ng_bdd)
while fsm_bdd.count_states(recur_bdd) > 0:
    pre_reach_bdd = new_bdd = fsm_bdd.pre(recur_bdd).intersection(ng_bdd)
    while fsm_bdd.count_states(new_bdd) > 0:
        pre_reach_bdd = pre_reach_bdd.union(new_bdd)
        if recur_bdd.entailed(pre_reach_bdd):
            # witness construction
            return (False, witness)
        new_bdd = (fsm_bdd.pre(new_bdd).diff(pre_reach_bdd)).intersection(ng_bdd)
    recur_bdd = recur_bdd.intersection(pre_reach_bdd)
return (True, None)
```

About `recur_bdd` and `new_bdd` initialization, the algorithm focuses on the subset of reachable states satisfying $f \wedge \neg g$, hence the intersection with `ng_bdd`. Furthermore, `pre_reach_bdd` is initialized to `new_bdd` as a simplification of the algorithm without affecting its correctness. The first iteration of the while loop will always include `new_bdd` in `pre_reach_bdd` anyway, and $\emptyset \cup \text{new_bdd} = \text{new_bdd}$.

Witness construction

The precondition to enter this portion of the code is that $f \wedge \neg g$ is repeatable. The first step to produce the witness at this point is to identify a loop head from the recurrent set computed above. At this stage in fact, `recur_bdd` contains all the states that can be reached infinitely often. Among these, there exists at least one state that can reach *itself* infinitely often, thus forming a loop.

To find such a state, the algorithm iteratively tests each state in `recur_bdd`. Starting from a candidate state, it repeatedly applies the post operator (restricted to `pre_reach_bdd`) to explore its successors (i.e., the set of states that can reach the portion of reachable states satisfying $f \wedge \neg g$) until no new states are found. If at any iteration the candidate state is included in the set of visited states, then it is selected as the loop head; otherwise, the algorithm proceeds to the next candidate in `recur_bdd`.

```
loop_head = recur_states_list.pop(0)
while True:
    Frontiers = []
    new_bdd = fsm_bdd.post(loop_head).intersection(pre_reach_bdd)
    R = new_bdd
    while fsm_bdd.count_states(new_bdd) > 0:
        R = R.union(new_bdd)
        Frontiers.append(R)
        new_bdd = fsm_bdd.post(new_bdd).intersection(pre_reach_bdd)
        new_bdd = new_bdd.diff(R)
    R = R.intersection(recur_bdd)
    if loop_head.entailed(R):
        break
    else:
        loop_head = recur_states_list.pop(0)
```

The solution is a direct translation of the pseudo-code presented during the course, modified as instructed to keep track of the sequence of frontiers useful to later reconstruct the loop body. `Frontiers` is initialized as empty and populated iteratively with layers. `Frontiers[i]` contains all the states that can be reached from the loop head in exactly i steps. The initialization of `R` to `new_bdd` is instead a pure simplification of the algorithm due to similar reasoning to the one used in the repeatability check, since the first iteration of the while loop will always include `new_bdd` in `R` anyway, and $\emptyset \cup \text{new_bdd} = \text{new_bdd}$.

Once the loop head has been identified, the algorithm proceeds to reconstruct the loop body. The goal is to extract one specific trace through the system that starts at some earlier state, progresses step by step, and eventually returns to the loop head, thereby forming a cycle.

The algorithm first identifies the smallest index k such that the loop head appears in `Frontiers[k]`, meaning there is a path of length k from the loop head back to itself. This defines the loop length (`k`). Starting from the loop head, the algorithm then reconstructs the loop by moving backward layer by layer, selecting at each step a predecessor of the current state that is in the correct frontier layer, while building the trace for visualization and the loop body. Below the python code that is strangely naturally divided by the end of the page:

```
k = 0
while not loop_head.entailed(Frontiers[k]):
    k += 1
```

```

loop = [loop_head]
curr = loop_head
trace = [loop_head.get_str_values()]
for i in range(k - 1, -1, -1):
    Pred = fsm_bdd.pre(curr).intersection(Frontiers[i])
    curr = fsm_bdd.pick_one_state(Pred)
    loop = [curr] + loop
    trace = [
        curr.get_str_values(),
        fsm_bdd.pick_one_inputs(
            fsm_bdd.get_inputs_between_states(curr, loop[0])
        ).get_str_values(),
    ] + trace

loop = [loop_head] + loop
trace = [
    loop_head.get_str_values(),
    fsm_bdd.pick_one_inputs(
        fsm_bdd.get_inputs_between_states(loop_head, loop[0])
    ).get_str_values(),
] + trace

```

The code above is actually a direct translation of the pseudo-code presented during the course, with the addition of the `trace` variable to store the concrete trace for visualization purposes. The trace is built by concatenating the string representation of the current state and the input that leads to the next state, ensuring that the trace reflects the actual transitions taken in the system.

The final step in constructing the counterexample witness involves generating a concrete path from an initial state to the previously identified loop head. The solution implements two sequential traversals:

- a backward search begins from the loop head and repeatedly applies the pre-image operator to symbolically compute the set of states that can reach the current set in one transition. This process continues until a state intersecting with the set of initial states is found, and the sequence of sets (stored in `images`) effectively encodes a path backward from the loop to the initial state space,
- To obtain a concrete, step-by-step trace, the forward traversal is applied to the computed `images`: at each layer, the intersection with the current reachable states and the post-image is used to extract one possible transition. A concrete state and the corresponding input leading to the next state are then appended to `trace_init`, reconstructing the prefix of the counterexample.

Finally, the prefix (`trace_init`) and the loop body (`trace`) are concatenated into a single sequence (`witness`).

```

images = list()
counterex = pre_counterex = loop_head
images.append(counterex)
while not pre_counterex.intersected(fsm_bdd.init):
    counterex = pre_counterex
    pre_counterex = fsm_bdd.pre(counterex)

```

```

images.insert(0, pre_counterex)

trace_init = list()
start = fsm_bdd.init
for i in range(0, len(images) - 1):
    start = start.intersection(images[i])
    post = fsm_bdd.post(start).intersection(images[i + 1])
    trace_init.append(fsm_bdd.pick_one_state(start).get_str_values())
    trace_init.append(
        fsm_bdd.pick_one_inputs(
            fsm_bdd.get_inputs_between_states(start, post)
        ).get_str_values()
    )
    start = post
witness = trace_init + trace

```

The backward-search and forward-extraction together outputs a prefix from an initial state to the loop head, by construction the result of the former is a sequence of BDDs

$$\text{images}[0], \dots, \text{images}[n]$$

such that

$$\text{images}[0] \cap \text{Init} \setminus \emptyset, \text{images}[n] = \text{loop_head}$$

and for each $0 \leq i < n$, every state in $\text{images}[i]$ has at least one successor in $\text{images}[i+1]$.

The forward-extraction of the prefix is driven by the list `images` built during the backward search. The starting point is initialized to the initial states, and then loop from 0 to `len(images)-1` performing the following steps:

1. Restrict the current reachable set to the i th layer:

```
start = start.intersection(images[i])
```

2. Compute the next-layer post-image:

```
post = fsm_bdd.post(start).intersection(images[i+1])
```

3. Extract and appending their string representations to `trace_init` one concrete state and the input leading it to the next layer:

```

trace_init.append(fsm_bdd.pick_one_state(start).get_str_values())
trace_init.append(fsm_bdd.pick_one_inputs(
    fsm_bdd.get_inputs_between_states(start, post)
).get_str_values())

```

4. Update `start` as the post variable in preparation for the next iteration:

```
start = post
```

At the end of the loop `trace_init` contains a concrete sequence

$$[s_0, a_0, s_1, a_1, \dots, s_{n-1}, a_{n-1}, s_n]$$

where $s_0 \in \text{images}[0] \cap \text{fsm_bdd.init}$ and $s_n = \text{loop_head}$, hence a looping execution in a form of a tuple of alternating states and inputs, starting and ending with a state, where the last state is in the sequence and indicates the starting point of the loop.

Finally `witness = trace_init + trace` concatenates the prefix with the loop body. At this point, `witness` contains a concrete counterexample trace that demonstrates the violation of the reactivity property. At this point the symbolic reactivity check function ends and returns `(False, witness)`.

Supervisory control

Uncontrolled plant design

The uncontrolled plant consists of five automata: the cabin, the door, and a button for each floor.

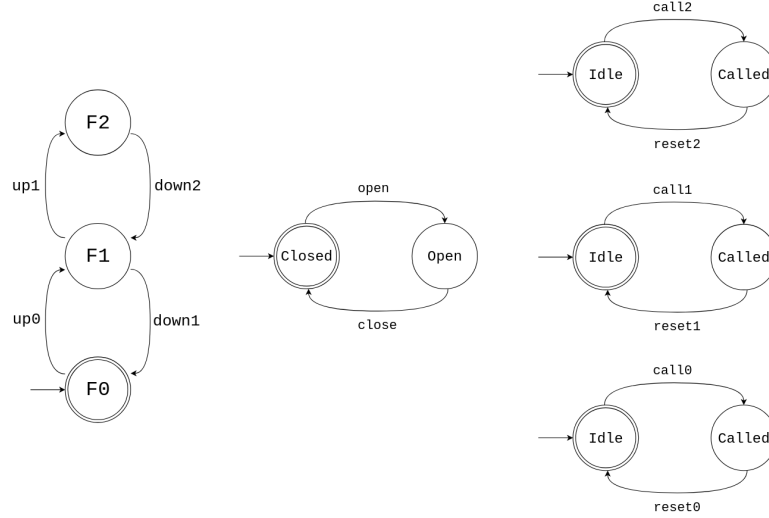


Figure 1: Cabin, door and buttons automata.

The cabin automaton has three states $\{F0, F1, F2\}$ representing the current floor of the cabin, the initial and only marked state is **F0**. The events $\{\text{up0}, \text{up1}, \text{down2}, \text{down1}\}$ of the automaton are all controllable and represent type of movement and source floor.

The door automaton has two states $\{\text{open}, \text{closed}\}$, the initial state and only marked state is **closed**, and two controllable events $\{\text{open}, \text{close}\}$ that allows the automaton to move between states.

The uncontrolled plant presents a button for each floor, each automaton has two states $\{\text{Idle}, \text{Called}\}$ and the initial and only marked one is **Idle**. Each automaton can transition to the **Called** state with the uncontrollable event call_i , and get back to **Idle** with the instead controllable event reset_i .

Requirements

The first requirement is to ensure that the cabin can only move when the door is closed. It can be expressed using temporal logic as follows:

$$\Box(\text{Door.Open} \rightarrow \neg(\text{Cabin.Moving}) \ \mathcal{U} \ \text{Door.Close})$$

Or, in other words, whenever its door is open the cabin must not move until the door is closed again.

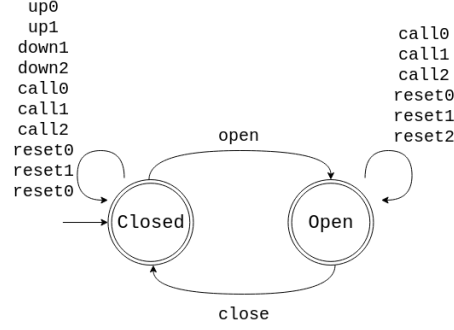


Figure 2: First requirement automaton.

The event set **Cabin.Moving** includes $\{\text{up0}, \text{up1}, \text{down1}, \text{down2}\}$, and in the proposed solution, the first requirement automaton (R1) disables them whenever the door is open.

The second requirement ensures that if the button corresponding to a floor is pressed, that floor is eventually served. Serving a floor is the process of opening and closing the door of the cabin. The implementation distinguishes between two situations for each floor: AtF_i and NotF_i , which indicate whether the cabin is at floor F_i or not, respectively.

- If the button i is called occurs while the cabin is in AtF_i , the floor is immediately served.
- If the call occurs while in NotF_i , a third state is used as a flag to indicate that the floor was requested while the cabin was away. In this case, service is triggered only when a movement event brings the cabin to the floor:
 - **down1** for F_0 ,
 - $\{\text{up0}, \text{down2}\}$ for F_1 ,
 - **up1** for F_2 .

The serving of a floor i consists of the following ordered sequence: first, the door is opened (**open**), then closed (**close**), and finally the button is reset (**reset_i**) to indicate the floor has been served. The automaton enforces this specific event order, while allowing the uncontrollable events **call_i** to occur at any time. A graphical representation of the second requirement is included below.

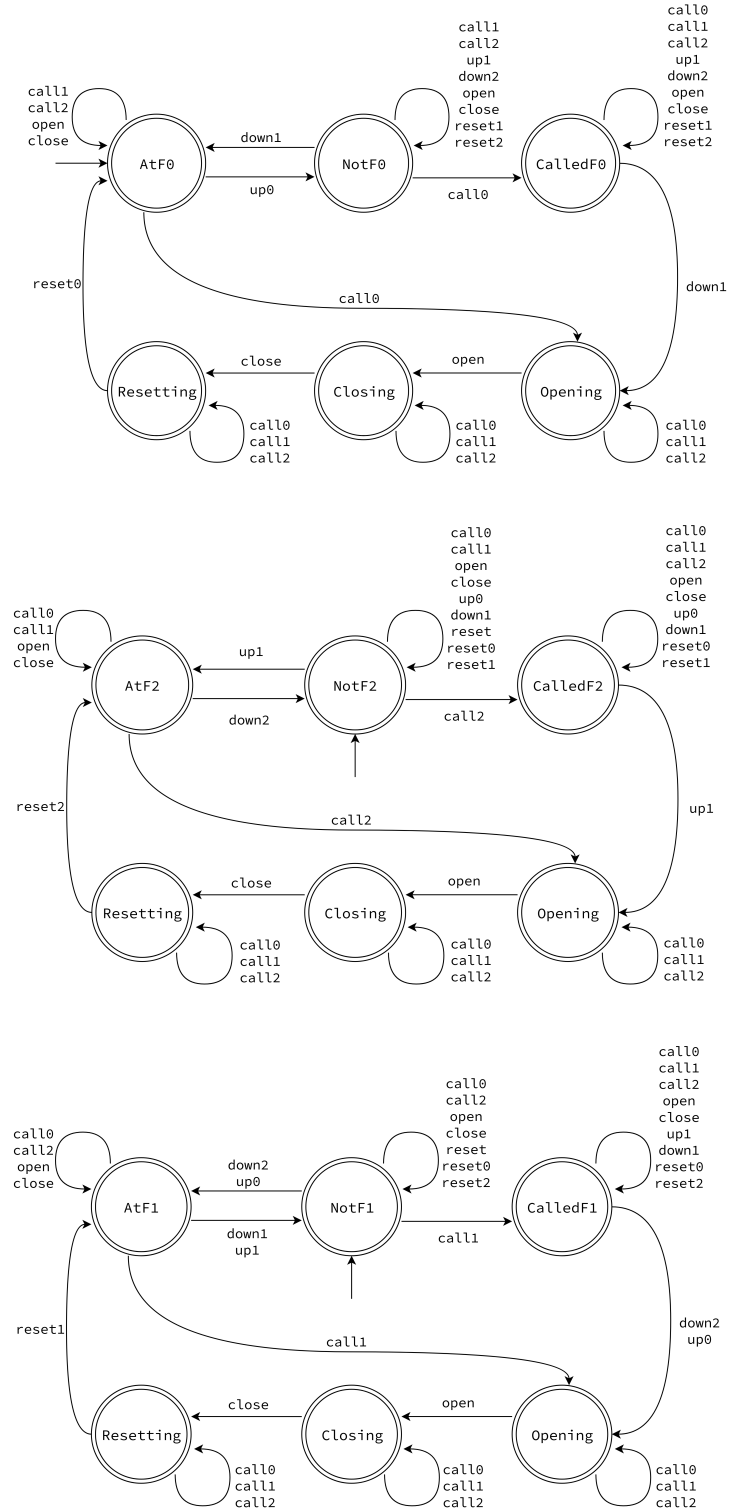


Figure 3: Second requirement automaton for first floor.

Marking and controllability choices

The cabin automaton uses states **F0**, **F1**, and **F2** to explicitly represent the cabin's position. The decision to make **F0** the only marked state is based on practical assumption since the absence of constraints by the homework text. Marking only **F0** also simplifies supervisory synthesis, as the plant must always be able to return to a marked state to ensure nonblocking behavior.

The door automaton's states **open** and **closed** are minimal yet sufficient to for the scope of the project. The controllability of **open** and **close** is essential for the first requirement other than reflecting the physical reality of an elevator door.

Each button automaton has a simple structure with two states: **Idle** and **Called**. The event call_i is modeled as uncontrollable since it is an input from the external plant. Conversely, reset_i is controllable because it is triggered by the elevator controller's logic after the system has served the corresponding floor.

Graphical interface

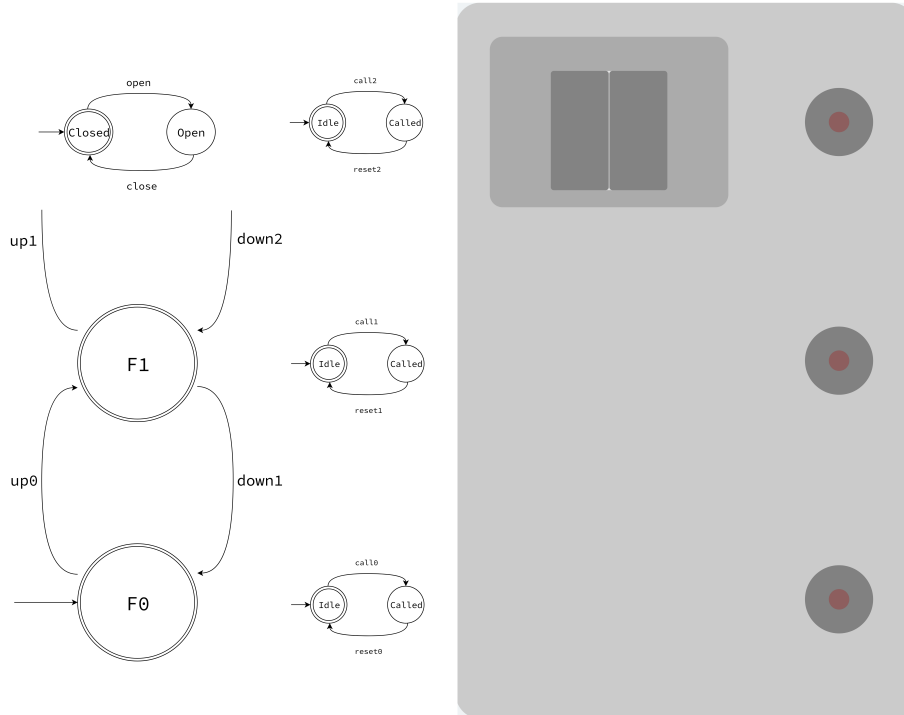


Figure 4: Graphical interface of the uncontrolled plant.

The SVG file for the graphical interface is manually adapted from an AI-generated output and provides a simple representation of the system. Manual adaptations included correcting element positions, adjusting colors to match automaton states, and ensuring all graphical components accurately reflect the system’s logic.

The cabin automaton controls the `y` attribute of the cabin element in the SVG, using a basic conditional to assign predefined altitude values corresponding to each floor. These values position the cabin appropriately within the graphical layout.

The cabin graphic element set includes both door sections. The `x` attributes of these sections are set to fixed values that reflect the current state of the door automaton, similarly to the cabin element, through conditional statements.

Each button is composed of two concentric circles: an outer, darker circle and an inner colored circle. The color of the inner circle indicates the current state of the corresponding button automaton and is managed by a conditional. A red inner circle represents the `Idle` state, while a green inner circle represents the `Called` state.

Controller synthesis

The controller synthesis process is performed using the `cifsupsynth` tool from the CIF (Compositional Interchange Format) toolset.

The synthesis begins by merging the uncontrolled plant with the two requirement automata (`R1` and `R2`) using `cifmerge`. Once merged, the `cifsupsynth` tool is used to generate the supervisor into `supervisor.cif` file. Finally, the controlled plant is obtained by merging the uncontrolled plant with the synthesized supervisor.